

LAPORAN PRAKTIKUM STRUKTUR DATA

DOUBLE LINKED LIST

Disusun Oleh:

Endy Pardilian 2511531017

Dosen Pengampu:

Wahyudi. Dr., S.T,M.T

Asisten Pratikum:

M Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data dengan materi Double Linked List (DLL) ini dapat diselesaikan dengan baik. Laporan ini membahas implementasi struktur data Double Linked List menggunakan bahasa pemrograman Java, mulai dari proses penambahan node, penelusuran data, penghapusan node, hingga penerapan DLL pada program playlist musik sederhana. Double Linked List merupakan pengembangan dari Single Linked List yang memiliki dua pointer, yaitu next dan prev, sehingga traversal data dapat dilakukan dua arah. Melalui praktikum ini, mahasiswa dapat memahami cara kerja serta implementasi operasi-operasi dasar pada struktur data Double Linked List. Diharapkan laporan ini dapat membantu dalam memahami konsep linked list dan penerapannya dalam pemrograman.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat.....	1
BAB II PEMBAHASAN.....	2
2.1 NodeDLL_2511531017.....	2
2.2 InsertDLL_2511531017.....	2
2.3 PenelusuranDLL_2511531017.....	7
2.4 HapusDLL_2511531017.....	9
TUGAS.....	13
2.5 Lagu_2511531017.....	13
2.6 Musik_2511531017.....	14
BAB III KESIMPULAN.....	22
3.1 Kesimpulan.....	22
3.2 Saran.....	22
DAFTAR PUSTAKA.....	23

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep penting dalam pemrograman yang digunakan untuk mengatur dan menyimpan data agar dapat diproses secara efisien. Salah satu struktur data linear yang sering digunakan adalah Linked List, yaitu struktur data yang menghubungkan setiap node menggunakan pointer. Pada praktikum ini dibahas mengenai Double Linked List (DLL), yaitu pengembangan dari Single Linked List yang memiliki dua pointer, yaitu *next* dan *prev*, sehingga traversal data dapat dilakukan ke arah maju maupun mundur. Implementasi Double Linked List sangat berguna dalam berbagai aplikasi, seperti playlist musik, navigasi data, dan sistem antrian.

Oleh karena itu, pemahaman mengenai operasi dasar seperti insert, traversal, dan delete pada Double Linked List sangat diperlukan dalam pembelajaran Struktur Data. Dalam praktikum ini, implementasi Double Linked List dilakukan menggunakan bahasa pemrograman Java dengan membuat beberapa program sederhana. Program yang dibuat meliputi proses penambahan node di awal, akhir, dan posisi tertentu, penelusuran data maju dan mundur, serta penghapusan node pada linked list. Selain itu, konsep Double Linked List juga diterapkan pada program playlist musik untuk menunjukkan penerapan struktur data dalam kehidupan sehari-hari. Dengan adanya praktikum ini, mahasiswa dapat memahami hubungan antar node serta cara kerja pointer *next* dan *prev* pada Double Linked List secara lebih jelas.

1.2 Tujuan

1. Memahami konsep dasar struktur data Double Linked List beserta hubungan antar node menggunakan pointer *next* dan *prev*.
2. Mengimplementasikan operasi dasar Double Linked List seperti insert, traversal, dan delete menggunakan bahasa pemrograman Java.
3. Menerapkan konsep Double Linked List pada studi kasus sederhana seperti program playlist musik.

1.3 Manfaat

1. Menambah pemahaman mengenai penggunaan struktur data linear dalam pemrograman.
2. Melatih kemampuan dalam membuat dan memanipulasi node pada Double Linked List menggunakan Java.
3. Membantu memahami penerapan Double Linked List dalam program yang digunakan pada kehidupan sehari-hari.

BAB II

PEMBAHASAN

2.1 NodeDLL_2511531017

```
package pekan6_2511531017;

public class NodeDLL_2511531017 {
    int data_1017;
    NodeDLL_2511531017 next_1017;
    NodeDLL_2511531017 prev_1017;

    public NodeDLL_2511531017(int data_1017) {
        this.data_1017 = data_1017;
        this.next_1017 = null;
        this.prev_1017 = null;
    }
}
```

Kode 2.1

Penjelasan:

```
NodeDLL_2511531017 next_1017;
NodeDLL_2511531017 prev_1017;
```

Digunakan untuk menyimpan referensi atau alamat node berikutnya dalam linked list.

```
public NodeDLL_2511531017(int data_1017) {
    this.data_1017 = data_1017;
    this.next_1017 = null;
    this.prev_1017 = null;
}
```

Constructor ini digunakan untuk membuat node baru dengan nilai data tertentu. Nilai parameter data_1017 akan disimpan ke dalam atribut data, sedangkan atribut next_1017 dan prev_1017 diinisialisasi dengan null karena node baru belum terhubung dengan node lain. Constructor ini mempermudah pembuatan node dalam Doubly Linked List. Dengan adanya constructor, proses inisialisasi node menjadi lebih praktis dan terstruktur.

Class berfungsi sebagai elemen dasar dalam struktur data Doubly Linked List yang menyimpan data serta referensi ke node sebelumnya dan berikutnya agar traversal dapat dilakukan dua arah.

2.2 InsertDLL_2511531017

```
package pekan6_2511531017;

public class InsertDLL_2511531017 {
    static NodeDLL_2511531017 insertBegin(NodeDLL_2511531017 head_1017, int
data_1017) {
        NodeDLL_2511531017 new_node_1017 = new
NodeDLL_2511531017(data_1017);
        new_node_1017.next_1017 = head_1017;
        if (head_1017 != null) {
            head_1017.prev_1017 = new_node_1017;
        }
    }
}
```

```

    }
    return new_node_1017;
}

public static NodeDLL_2511531017 insertEnd (NodeDLL_2511531017
head_1017, int newData_1017) {
    NodeDLL_2511531017 newNode_1017 = new
NodeDLL_2511531017(newData_1017);
    if (head_1017 == null) {
        head_1017 = newNode_1017;
    }else {
        NodeDLL_2511531017 curr_1017 = head_1017;
        while (curr_1017.next_1017 != null) {
            curr_1017 = curr_1017.next_1017;
        }
        curr_1017.next_1017 = newNode_1017;
        newNode_1017.prev_1017 = curr_1017;
    }
    return head_1017;
}

public static NodeDLL_2511531017 insertAtPosition (NodeDLL_2511531017
head_1017, int pos_1017, int new_data_1017) {
    NodeDLL_2511531017 new_node_1017 = new
NodeDLL_2511531017(new_data_1017);
    if (pos_1017 == 1) {
        new_node_1017.next_1017 = head_1017;
        if (head_1017 != null) {
            head_1017.prev_1017 = new_node_1017;
        }
        head_1017 = new_node_1017;
        return head_1017;
    }
    NodeDLL_2511531017 curr_1017 = head_1017;
    for (int i = 1; i < pos_1017 - 1 && curr_1017 != null; ++i) {
        curr_1017 = curr_1017.next_1017;
    }
    if(curr_1017 == null) {
        System.out.println("Posisi tidak ada");
        return head_1017;
    }
    new_node_1017.prev_1017 = curr_1017;
    new_node_1017.next_1017 = curr_1017.next_1017;
    curr_1017.next_1017 = new_node_1017;
    if (new_node_1017.next_1017 != null) {
        new_node_1017.next_1017.prev_1017 = new_node_1017;
    }
    return head_1017;
}

public static void printList (NodeDLL_2511531017 head_1017) {
    NodeDLL_2511531017 curr_1017 = head_1017;
    while (curr_1017 != null) {
        System.out.print(curr_1017.data_1017 + " <-> ");
        curr_1017 = curr_1017.next_1017;
    }
    System.out.println();
}

public static void main(String[] args) {
    // TODO Auto-generated method stub
    NodeDLL_2511531017 head_1017 = new NodeDLL_2511531017(2);
    head_1017.next_1017 = new NodeDLL_2511531017 (3);
    head_1017.next_1017.prev_1017 = head_1017;
}

```

```

        head_1017.next_1017.next_1017 = new NodeDLL_2511531017(5);
        head_1017.next_1017.next_1017.prev_1017 = head_1017.next_1017;
        System.out.print("DLL awal: ");
        printList (head_1017);
        head_1017 = insertBegin (head_1017, 1);
        System.out.print("simpul 1 ditambah di awal: ");
        printList(head_1017);
        System.out.print("simpul 6 ditambah di akhir: ");
        int data_1017 = 6;
        head_1017 = insertEnd( head_1017, data_1017);
        printList(head_1017);
        System.out.print("tambah node 4 di posisi 4:");
        int data2_1017 = 4;
        int pos_1017 = 4;
        head_1017 = insertAtPosition(head_1017, pos_1017, data2_1017);
        printList(head_1017);
    }
}

```

Kode 2.2

Penjelasan:

```

static NodeDLL_2511531017 insertBegin(NodeDLL_2511531017 head_1017, int
data_1017) {
    NodeDLL_2511531017 new_node_1017 = new
NodeDLL_2511531017(data_1017);
    new_node_1017.next_1017 = head_1017;
    if (head_1017 != null) {
        head_1017.prev_1017 = new_node_1017;
    }
    return new_node_1017;
}

```

Method ini digunakan untuk menambahkan node di bagian awal Double Linked List. Node baru akan dibuat lalu dihubungkan ke head lama melalui atribut next. Jika list tidak kosong, maka prev pada head lama akan diarahkan ke node baru. Setelah itu, node baru dikembalikan sebagai head yang baru. Method ini mempermudah proses penambahan data di awal list tanpa perlu traversal.

```

public static NodeDLL_2511531017 insertEnd (NodeDLL_2511531017 head_1017, int
newData_1017) {
    NodeDLL_2511531017 newNode_1017 = new
NodeDLL_2511531017(newData_1017);
    if (head_1017 == null) {
        head_1017 = newNode_1017;
    }else {
        NodeDLL_2511531017 curr_1017 = head_1017;
        while (curr_1017.next_1017 != null) {
            curr_1017 = curr_1017.next_1017;
        }
        curr_1017.next_1017 = newNode_1017;
        newNode_1017.prev_1017 = curr_1017;
    }
    return head_1017;
}

```

Method ini digunakan untuk menambahkan node di bagian akhir linked list. Program akan menelusuri node hingga mencapai node terakhir, kemudian menghubungkan node

baru di bagian akhir. Atribut prev pada node baru akan menunjuk ke node sebelumnya sehingga hubungan dua arah tetap terjaga. Jika list kosong, node baru langsung menjadi head. Method ini memastikan struktur DLL tetap terhubung dengan benar.

```
public static NodeDLL_2511531017 insertAtPosition (NodeDLL_2511531017
head_1017, int pos_1017, int new_data_1017) {
    NodeDLL_2511531017 new_node_1017 = new
NodeDLL_2511531017(new_data_1017);
    if (pos_1017 == 1) {
        new_node_1017.next_1017 = head_1017;
        if (head_1017 != null) {
            head_1017.prev_1017 = new_node_1017;
        }
        head_1017 = new_node_1017;
        return head_1017;
    }
    NodeDLL_2511531017 curr_1017 = head_1017;
    for (int i = 1; i < pos_1017 - 1 && curr_1017 != null; ++i) {
        curr_1017 = curr_1017.next_1017;
    }
    if (curr_1017 == null) {
        System.out.println("Posisi tidak ada");
        return head_1017;
    }
    new_node_1017.prev_1017 = curr_1017;
    new_node_1017.next_1017 = curr_1017.next_1017;
    curr_1017.next_1017 = new_node_1017;
    if (new_node_1017.next_1017 != null) {
        new_node_1017.next_1017.prev_1017 = new_node_1017;
    }
    return head_1017;
}
```

Method ini digunakan untuk menambahkan node pada posisi tertentu dalam linked list. Program akan menelusuri node hingga posisi yang diinginkan, kemudian menyisipkan node baru di antara node sebelumnya dan node sesudahnya. Relasi next dan prev akan diperbarui agar hubungan antar node tetap benar. Jika posisi tidak ditemukan, program akan menampilkan pesan error. Method ini mempermudah manipulasi data pada posisi tertentu.

```
public static void printList (NodeDLL_2511531017 head_1017) {
    NodeDLL_2511531017 curr_1017 = head_1017;
    while (curr_1017 != null) {
        System.out.print(curr_1017.data_1017 + " <-> ");
        curr_1017 = curr_1017.next_1017;
    }
    System.out.println();
}
```

Method ini digunakan untuk menampilkan seluruh isi Double Linked List dari awal hingga akhir. Program akan melakukan traversal menggunakan pointer next dan mencetak data setiap node. Simbol <-> digunakan untuk menunjukkan hubungan dua arah antar node dalam DLL. Method ini membantu pengguna melihat isi dan urutan data dalam list. Jika traversal selesai, program akan berpindah ke baris baru.

```
public static void main(String[] args) {
    // TODO Auto-generated method stub
    NodeDLL_2511531017 head_1017 = new NodeDLL_2511531017(2);
    head_1017.next_1017 = new NodeDLL_2511531017 (3);
    head_1017.next_1017.prev_1017 = head_1017;
    head_1017.next_1017.next_1017 = new NodeDLL_2511531017(5);
    head_1017.next_1017.next_1017.prev_1017 = head_1017.next_1017;
    System.out.print("DLL awal: ");
    printList (head_1017);
    head_1017 = insertBegin (head_1017, 1);
}
```



```

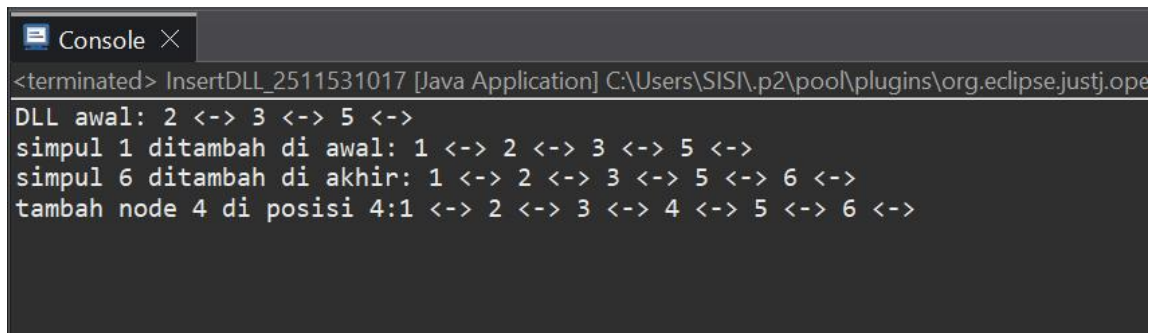
System.out.print("simpul 1 ditambah di awal: ");
printList(head_1017);
System.out.print("simpul 6 ditambah di akhir: ");
int data_1017 = 6;
head_1017 = insertEnd( head_1017, data_1017);
printList(head_1017);
System.out.print("tambah node 4 di posisi 4:");
int data2_1017 = 4;
int pos_1017 = 4;
head_1017 = insertAtPosition(head_1017, pos_1017, data2_1017);
printList(head_1017);

```

Method ini digunakan untuk menjalankan dan menguji operasi pada Double Linked List. Program membuat list awal, kemudian melakukan penambahan node di awal, akhir, dan posisi tertentu. Setelah setiap operasi, isi linked list akan ditampilkan menggunakan method printList. Method ini membantu memastikan bahwa setiap operasi insert berjalan dengan benar. Selain itu, pengguna dapat melihat perubahan struktur DLL setelah setiap proses penambahan data.

Program digunakan untuk mengimplementasikan operasi penambahan node pada struktur data Double Linked List (DLL). Saat dijalankan, program akan membuat linked list awal berisi data 2, 3, dan 5, kemudian menampilkan isi list tersebut. Setelah itu program melakukan penambahan node di awal, di akhir, dan pada posisi tertentu sehingga isi linked list terus berubah dan ditampilkan setiap selesai proses insert. Program ini menunjukkan cara kerja Double Linked List dengan memanfaatkan hubungan dua arah antar node melalui pointer next dan prev.

Output:



```

Console X
<terminated> InsertDLL_2511531017 [Java Application] C:\Users\SISI\p2\pool\plugins\org.eclipse.justj.open
DLL awal: 2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->

```

Gambar 2.2

2.3 PenelusuranDLL_2511531017

```
package pekan6_2511531017;
```

```

public class PenulusuranDLL_2511531017 {
    static void forwardTraversal (NodeDLL_2511531017 head_1017) {
        NodeDLL_2511531017 curr_1017 = head_1017;
        while (curr_1017 != null) {
            System.out.print(curr_1017.data_1017 + " <-> ");
            curr_1017 = curr_1017.next_1017;
        }
        System.out.println();
    }
    static void backwardTraversal (NodeDLL_2511531017 tail_1017) {
        NodeDLL_2511531017 curr_1017 = tail_1017;
        while (curr_1017 != null) {
            System.out.print(curr_1017.data_1017 + " <-> ");
            curr_1017 = curr_1017.prev_1017;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        NodeDLL_2511531017 head_1017 = new NodeDLL_2511531017(1);
        NodeDLL_2511531017 second_1017 = new NodeDLL_2511531017(2);
        NodeDLL_2511531017 third_1017 = new NodeDLL_2511531017(3);

        head_1017.next_1017 = second_1017;
        second_1017.prev_1017 = head_1017;
        second_1017.next_1017 = third_1017;
        third_1017.prev_1017 = second_1017;

        System.out.println("Penulusuran maju:");
        forwardTraversal(head_1017);
        System.out.println("Penulusuran mundur:");
        backwardTraversal(third_1017);
    }
}

```

Kode 2.3

Penjelasan:

```

static void forwardTraversal (NodeDLL_2511531017 head_1017) {
    NodeDLL_2511531017 curr_1017 = head_1017;
    while (curr_1017 != null) {
        System.out.print(curr_1017.data_1017 + " <-> ");
        curr_1017 = curr_1017.next_1017;
    }
    System.out.println();
}

```

Method ini digunakan untuk melakukan penelusuran data dari depan ke belakang pada Double Linked List. Program akan memulai traversal dari node head, kemudian bergerak ke node berikutnya menggunakan pointer next. Setiap data node akan ditampilkan hingga mencapai node terakhir yang bernilai null. Method ini menunjukkan proses traversal maju pada DLL.

```

static void backwardTraversal (NodeDLL_2511531017 tail_1017) {

```

```

NodeDLL_2511531017 curr_1017 = tail_1017;
while (curr_1017 != null) {
    System.out.print(curr_1017.data_1017 + " <-> ");
    curr_1017 = curr_1017.prev_1017;
}
System.out.println();

```

Method ini digunakan untuk melakukan penelusuran data dari belakang ke depan pada Double Linked List. Traversal dimulai dari node terakhir (tail), kemudian bergerak ke node sebelumnya menggunakan pointer prev. Data setiap node akan ditampilkan hingga mencapai node pertama. Method ini menunjukkan kelebihan DLL yang dapat ditelusuri dua arah.

```

public static void main(String[] args) {
    // TODO Auto-generated method stub
    NodeDLL_2511531017 head_1017 = new NodeDLL_2511531017(1);
    NodeDLL_2511531017 second_1017 = new NodeDLL_2511531017(2);
    NodeDLL_2511531017 third_1017 = new NodeDLL_2511531017(3);
    head_1017.next_1017 = second_1017;
    second_1017.prev_1017 = head_1017;
    second_1017.next_1017 = third_1017;
    third_1017.prev_1017 = second_1017;
    System.out.println("Penelusuran maju:");
    forwardTraversal(head_1017);
    System.out.println("Penelusuran mundur:");
    backwardTraversal(third_1017);
}

```

Method ini digunakan untuk membuat dan menghubungkan beberapa node dalam Double Linked List. Program membuat tiga node dengan data 1, 2, dan 3, lalu menghubungkannya menggunakan pointer next dan prev. Setelah itu program menjalankan traversal maju dan traversal mundur untuk menampilkan isi linked list dari dua arah. Hasil output menunjukkan bahwa data dapat diakses dari depan maupun belakang.

Program PenelusuranDLL_2511531017 digunakan untuk menunjukkan proses traversal pada Doubly Linked List ke arah maju dan mundur. Saat dijalankan, program akan menampilkan data dari node pertama ke terakhir menggunakan pointer next, kemudian menampilkan data dari node terakhir ke pertama menggunakan pointer prev. Program ini membuktikan bahwa Doubly Linked List memiliki kemampuan traversal dua arah yang tidak dimiliki Single Linked List.

Output:

```

<terminated> PenelusuranDLL_2511531017 [Jav
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->

```

2.4 HapusDLL_2511531017

```

package pekan6_2511531017;

```

```

public class HapusDLL_2511531017 {
    public static NodeDLL_2511531017 delHead_1017(NodeDLL_2511531017
head_1017) {
        if (head_1017 == null) {
            return null; }
        NodeDLL_2511531017 temp_1017 = head_1017;
        head_1017 = head_1017.next_1017;
        if (head_1017 != null) {
            head_1017.prev_1017 = null; }
        return head_1017;
    }

    public static NodeDLL_2511531017 delLast (NodeDLL_2511531017 head_1017)
{
        if (head_1017 == null) {
            return null; }
        if (head_1017.next_1017 == null) {
            return null; }
        NodeDLL_2511531017 curr_1017 = head_1017;
        while (curr_1017.next_1017 != null) {
            curr_1017 = curr_1017.next_1017;
        }
        if (curr_1017.prev_1017 != null) {
            curr_1017.prev_1017.next_1017 = null; }
        return head_1017;
    }

    public static NodeDLL_2511531017 delPos_1017(NodeDLL_2511531017
head_1017, int pos_1017) {
        if (head_1017 == null) {
            return head_1017; }
        NodeDLL_2511531017 curr_1017 = head_1017;
        for (int i = 1; curr_1017 != null && i < pos_1017; ++i) {
            curr_1017 = curr_1017.next_1017; }
        if (curr_1017 == null) {
            return head_1017; }
        if (curr_1017.prev_1017 != null) {
            curr_1017.prev_1017.next_1017 = curr_1017.next_1017; }
        if (curr_1017.next_1017 != null) {
            curr_1017.next_1017.prev_1017 = curr_1017.prev_1017; }
        if (head_1017 == curr_1017) {
            head_1017 = curr_1017.next_1017; }
        return head_1017;
    }

    public static void printList (NodeDLL_2511531017 head_1017) {
        NodeDLL_2511531017 curr_1017 = head_1017;
        while (curr_1017 != null) {
            System.out.print(curr_1017.data_1017 + " ");
            curr_1017 = curr_1017.next_1017;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        NodeDLL_2511531017 head_1017 = new NodeDLL_2511531017(1);
        head_1017.next_1017 = new NodeDLL_2511531017(2);
        head_1017.next_1017.prev_1017 = head_1017;
    }
}

```

```

        head_1017.next_1017.next_1017 = new NodeDLL_2511531017(3);
        head_1017.next_1017.next_1017.prev_1017 = head_1017.next_1017;
        head_1017.next_1017.next_1017.next_1017 = new
NodeDLL_2511531017(4);
        head_1017.next_1017.next_1017.next_1017.prev_1017
= head_1017.next_1017.next_1017;
        head_1017.next_1017.next_1017.next_1017.next_1017 = new
NodeDLL_2511531017(5);
        head_1017.next_1017.next_1017.next_1017.next_1017.prev_1017
= head_1017.next_1017.next_1017.next_1017;
        System.out.print("DLL Awal: ");
        printList(head_1017);
        System.out.print("Setelah head dihapus: ");
        head_1017 = delHead_1017(head_1017);
        printList(head_1017);
        System.out.print("Setelah node terakhir dihapus: ");
        head_1017 = delHead_1017(head_1017);
        printList(head_1017);
        System.out.print("menghapus node terakhir dihapus: ");
        head_1017 = delHead_1017(head_1017);
        head_1017 = delPos_1017(head_1017, 2);
        printList(head_1017);
    }
}

```

Kode 2.4

Penjelasan:

```

public static NodeDLL_2511531017 delHead_1017(NodeDLL_2511531017 head_1017) {
    if (head_1017 == null) {
        return null;
    }
    NodeDLL_2511531017 temp_1017 = head_1017;
    head_1017 = head_1017.next_1017;
    if (head_1017 != null) {
        head_1017.prev_1017 = null;
    }
    return head_1017;
}

```

Method ini digunakan untuk menghapus node pertama (head) pada Double Linked List. Setelah node pertama dihapus, head akan dipindahkan ke node berikutnya dan pointer prev pada head baru diubah menjadi null. Jika linked list kosong, method akan mengembalikan null. Method ini mempermudah proses penghapusan data di bagian depan list.

```

public static NodeDLL_2511531017 delLast (NodeDLL_2511531017 head_1017) {
    if (head_1017 == null) {
        return null;
    }
    if (head_1017.next_1017 == null) {
        return null;
    }
    NodeDLL_2511531017 curr_1017 = head_1017;
    while (curr_1017.next_1017 != null) {
        curr_1017 = curr_1017.next_1017;
    }
    if (curr_1017.prev_1017 != null) {

```

```
curr_1017.prev_1017.next_1017 = null;}  
return head_1017;
```

Method ini digunakan untuk menghapus node terakhir pada Double Linked List. Program akan melakukan traversal hingga mencapai node terakhir, kemudian hubungan node sebelumnya diputus dengan mengubah pointer next menjadi null. Jika list kosong atau hanya memiliki satu node, method akan mengembalikan null. Method ini digunakan untuk menghapus data pada bagian akhir linked list.

```
public static NodeDLL_2511531017 delPos_1017(NodeDLL_2511531017 head_1017,  
int pos_1017) {  
    if (head_1017 == null) {  
        return head_1017;}  
    NodeDLL_2511531017 curr_1017 = head_1017;  
    for (int i = 1; curr_1017 != null && i < pos_1017; ++i) {  
        curr_1017 = curr_1017.next_1017;}  
    if (curr_1017 == null) {  
        return head_1017;}  
    if (curr_1017.prev_1017 != null) {  
        curr_1017.prev_1017.next_1017 = curr_1017.next_1017;}  
    if (curr_1017.next_1017 != null) {  
        curr_1017.next_1017.prev_1017 = curr_1017.prev_1017;}  
    if (head_1017 == curr_1017) {  
        head_1017 = curr_1017.next_1017;}  
    return head_1017;
```

Method ini digunakan untuk menghapus node pada posisi tertentu dalam linked list. Program akan menelusuri node hingga posisi yang diinginkan, kemudian memperbarui pointer next dan prev agar hubungan antar node tetap benar setelah penghapusan dilakukan. Jika posisi tidak ditemukan, linked list tidak akan berubah. Method ini mempermudah manipulasi data pada posisi tertentu.

```
public static void printList (NodeDLL_2511531017 head_1017) {  
    NodeDLL_2511531017 curr_1017 = head_1017;  
    while (curr_1017 != null) {  
        System.out.print(curr_1017.data_1017 + " ");  
        curr_1017 = curr_1017.next_1017;  
    }  
    System.out.println();
```

Method ini digunakan untuk menampilkan seluruh isi Double Linked List dari awal hingga akhir. Program akan melakukan traversal menggunakan pointer next dan mencetak data setiap node secara berurutan. Method ini membantu pengguna melihat perubahan isi linked list setelah proses penghapusan dilakukan.

```
public static void main(String[] args) {  
    // TODO Auto-generated method stub  
    NodeDLL_2511531017 head_1017 = new NodeDLL_2511531017(1);  
    head_1017.next_1017 = new NodeDLL_2511531017(2);  
    head_1017.next_1017.prev_1017 = head_1017;  
    head_1017.next_1017.next_1017 = new NodeDLL_2511531017(3);  
    head_1017.next_1017.next_1017.prev_1017 = head_1017.next_1017;  
    head_1017.next_1017.next_1017.next_1017 = new  
NodeDLL_2511531017(4);  
    head_1017.next_1017.next_1017.next_1017.prev_1017  
=head_1017.next_1017.next_1017;
```

```

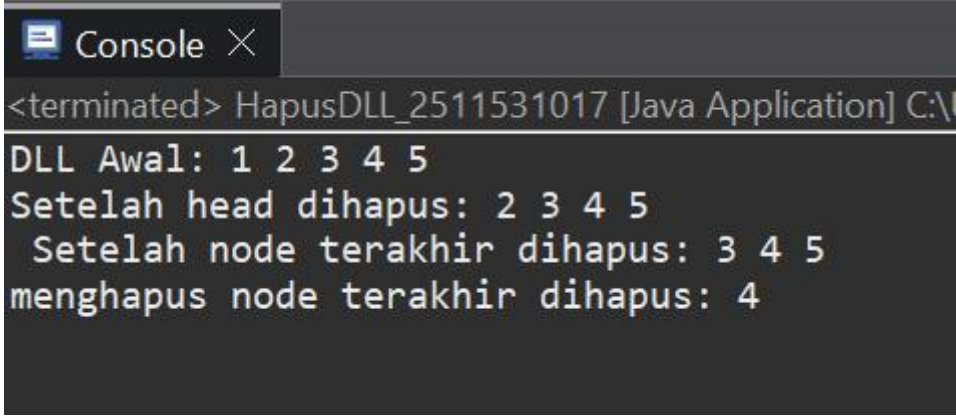
        head_1017.next_1017.next_1017.next_1017.next_1017 = new
NodeDLL_2511531017(5);
        head_1017.next_1017.next_1017.next_1017.next_1017.prev_1017
=head_1017.next_1017.next_1017.next_1017;
        System.out.print("DLL Awal: ");
        printList(head_1017);
        System.out.print("Setelah head dihapus: ");
        head_1017 = delHead_1017(head_1017);
        printList(head_1017);
        System.out.print(" Setelah node terakhir dihapus: ");
        head_1017 = delHead_1017(head_1017);
        printList(head_1017);
        System.out.print("menghapus node terakhir dihapus: ");
        head_1017 = delHead_1017(head_1017);
        head_1017 = delPos_1017(head_1017, 2);
        printList(head_1017);

```

Method ini digunakan untuk membuat linked list awal dan menguji operasi penghapusan node pada Double Linked List. Program membuat linked list berisi data 1, 2, 3, 4, dan 5, kemudian menampilkan isi list awal. Setelah itu dilakukan penghapusan node di bagian head dan posisi tertentu, lalu hasil perubahan linked list ditampilkan menggunakan method printList. Method ini membantu memastikan bahwa proses penghapusan node berjalan dengan benar.

Program digunakan untuk mengimplementasikan operasi penghapusan node pada Double Linked List. Saat dijalankan, program akan membuat linked list awal kemudian melakukan beberapa proses delete seperti menghapus node pertama, node terakhir, dan node pada posisi tertentu. Setelah setiap proses penghapusan, isi linked list akan ditampilkan agar perubahan struktur data dapat terlihat secara langsung.

Output:



```

Console X
<terminated> HapusDLL_2511531017 [Java Application] C:\
DLL Awal: 1 2 3 4 5
Setelah head dihapus: 2 3 4 5
  Setelah node terakhir dihapus: 3 4 5
menghapus node terakhir dihapus: 4

```

Gambar 2.4

TUGAS

2.5 Lagu_2511531017

```

package pekan6_2511531017;

public class Lagu_2511531017 {
    String judul_1017;
    String penyanyi_1017;
    Lagu_2511531017 next_1017;
    Lagu_2511531017 prev_1017;
    public Lagu_2511531017(String judul_1017, String penyanyi_1017) {
        this.judul_1017 = judul_1017;
        this.penyanyi_1017 = penyanyi_1017;
        this.next_1017 = null;
        this.prev_1017 = null;
    }
    public String getJudul_1017() {
        return judul_1017;
    }
    public String getPenyanyi_1017() {
        return penyanyi_1017;
    }
    public Lagu_2511531017 getNext_1017() {
        return next_1017;
    }
    public Lagu_2511531017 getPrev_1017() {
        return prev_1017;
    }
    public void setNext_1017(Lagu_2511531017 next_1017) {
        this.next_1017 = next_1017;
    }
    public void setPrev_1017(Lagu_2511531017 prev_1017) {
        this.prev_1017 = prev_1017;
    }
}

```

Kode 2.5

Penjelasan:

```

public Lagu_2511531017(String judul_1017, String penyanyi_1017) {
    this.judul_1017 = judul_1017;
    this.penyanyi_1017 = penyanyi_1017;
    this.next_1017 = null;
    this.prev_1017 = null;
}

```

Constructor ini digunakan untuk membuat node lagu baru dengan mengisi data judul dan penyanyi. Selain itu, pointer next dan prev diatur bernilai null karena node belum terhubung dengan node lain.

```

public String getJudul_1017() {
    return judul_1017;
}
public String getPenyanyi_1017() {
    return penyanyi_1017;
}
public Lagu_2511531017 getNext_1017() {
    return next_1017;
}
public Lagu_2511531017 getPrev_1017() {
    return prev_1017;
}

```


Method-method getter ini digunakan untuk mengambil atau mengembalikan nilai atribut yang terdapat pada class Lagu_2511531017. Method getJudul_1017() digunakan untuk mengambil judul lagu, getPenyanyi_1017() digunakan untuk mengambil nama penyanyi, sedangkan getNext_1017() dan getPrev_1017() digunakan untuk mengambil referensi node berikutnya dan sebelumnya dalam Double Linked List. Getter mempermudah akses data tanpa mengubah isi atribut yang ada pada objek.

```
public void setNext_1017(Lagu_2511531017 next_1017) {
    this.next_1017 = next_1017;
}
```

Method ini digunakan untuk menghubungkan node saat ini dengan node berikutnya melalui pointer next. Method setter membantu pembentukan hubungan antar node pada DLL.

```
public void setPrev_1017(Lagu_2511531017 prev_1017) {
    this.prev_1017 = prev_1017;
}
```

Method ini digunakan untuk menghubungkan node saat ini dengan node sebelumnya melalui pointer prev. Dengan method ini, hubungan dua arah antar node dapat terbentuk dengan benar.

Class berfungsi sebagai node pada struktur data Double Linked List yang digunakan untuk menyimpan data lagu dalam playlist musik. Setiap node menyimpan informasi judul lagu, nama penyanyi, serta hubungan ke node berikutnya (next) dan sebelumnya (prev) sehingga playlist dapat ditelusuri dua arah.\

2.6 Musik_2511531017

```
package pekan6_2511531017;
import java.util.*;
public class Musik_2511531017 {

    Lagu_2511531017 head_1017 = null;
    Lagu_2511531017 tail_1017 = null;
    public void tambahLagu_1017(String judul_1017, String penyanyi_1017) {
        Lagu_2511531017 baru_1017 = new Lagu_2511531017(judul_1017,
        penyanyi_1017);
        if (head_1017 == null) {
            head_1017 = baru_1017;
            tail_1017 = baru_1017;
        } else {
            tail_1017.next_1017 = baru_1017;
            baru_1017.prev_1017 = tail_1017;

            tail_1017 = baru_1017;
        }
        System.out.println("Lagu berhasil ditambahkan!");
    }

    public void hapusLaguAwal_1017() {
        if (head_1017 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
    }
}
```

```

        System.out.println("Lagu " + head_1017.getJudul_1017() + " berhasil
dihapus");
        head_1017 = head_1017.next_1017;
        if (head_1017 != null) {
            head_1017.prev_1017 = null;
        } else {
            tail_1017 = null;
        }
    }

    public void tampilMaju_1017() {
        if (head_1017 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        Lagu_2511531017 curr_1017 = head_1017;
        System.out.println("=== Playlist Maju ===");
        while (curr_1017 != null) {
            System.out.println(curr_1017.getJudul_1017() + " - " +
curr_1017.getPenyanyi_1017());
            curr_1017 = curr_1017.next_1017;
        }
    }

    public void tampilMundur_1017() {
        if (tail_1017 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        Lagu_2511531017 curr_1017 = tail_1017;
        System.out.println("=== Playlist Mundur ===");
        while (curr_1017 != null) {
            System.out.println(curr_1017.getJudul_1017() + " - " +
curr_1017.getPenyanyi_1017());
            curr_1017 = curr_1017.prev_1017;
        }
    }

    public void cariLagu_1017(String judul_1017) {
        if (head_1017 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        Lagu_2511531017 curr_1017 = head_1017;
        boolean ketemu_1017 = false;
        while (curr_1017 != null) {
            if (curr_1017.getJudul_1017().equalsIgnoreCase(judul_1017)) {
                System.out.println("Lagu ditemukan!");
                System.out.println("Judul : " + curr_1017.getJudul_1017());
                System.out.println("Penyanyi : " +
curr_1017.getPenyanyi_1017());
                ketemu_1017 = true;
                break;
            }
            curr_1017 = curr_1017.next_1017;
        }
        if (!ketemu_1017) {
            System.out.println("Lagu tidak ditemukan!");
        }
    }

```

```

    }

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Musik_2511531017 musik_1017 = new Musik_2511531017();
        int pilihan_1017;
        do {
            System.out.println("\n=== Playlist Musik NIM: 2511531017 ===");
            System.out.println("1. Tambah Lagu");
            System.out.println("2. Hapus Lagu Pertama");
            System.out.println("3. Lihat Playlist (Maju)");
            System.out.println("4. Lihat Playlist (Mundur)");
            System.out.println("5. Cari Lagu");
            System.out.println("6. Keluar");
            System.out.print("Pilihan: ");
            pilihan_1017 = input.nextInt();
            input.nextLine();
            switch (pilihan_1017) {
                case 1:
                    System.out.print("Judul Lagu: ");
                    String judul_1017 = input.nextLine();
                    System.out.print("Penyanyi: ");
                    String penyanyi_1017 = input.nextLine();
                    musik_1017.tambahLagu_1017(
                        judul_1017,
                        penyanyi_1017);
                    break;
                case 2:
                    musik_1017.hapusLaguAwal_1017();
                    break;
                case 3:
                    musik_1017.tampilMaju_1017();
                    break;
                case 4:
                    musik_1017.tampilMundur_1017();
                    break;
                case 5:
                    System.out.print("Masukkan judul lagu: ");
                    String cari_1017 = input.nextLine();
                    musik_1017.cariLagu_1017(cari_1017);
                    break;
                case 6:
                    System.out.println("Program selesai");
                    break;
                default:
                    System.out.println("Pilihan tidak valid!");
            }
        } while (pilihan_1017 != 6);
        input.close();
    }
}

```

Kode 2.6

Penjelasan:

```

public void tambahLagu_1017(String judul_1017, String penyanyi_1017) {

```

```

        Lagu_2511531017 baru_1017 = new Lagu_2511531017(judul_1017,
penyanyi_1017);
        if (head_1017 == null) {
            head_1017 = baru_1017;
            tail_1017 = baru_1017;
        } else {
            tail_1017.next_1017 = baru_1017;
            baru_1017.prev_1017 = tail_1017;

            tail_1017 = baru_1017;
        }
        System.out.println("Lagu berhasil ditambahkan!");
    }

```

Method ini digunakan untuk menambahkan lagu baru ke bagian akhir playlist. Jika playlist masih kosong maka node baru akan menjadi head dan tail, sedangkan jika playlist sudah memiliki data maka node baru akan dihubungkan ke node terakhir menggunakan pointer next dan prev. Setelah lagu berhasil ditambahkan, program akan menampilkan pesan bahwa lagu berhasil dimasukkan ke playlist.

```

public void hapusLaguAwal_1017() {
    if (head_1017 == null) {
        System.out.println("Playlist kosong!");
        return;
    }
    System.out.println("Lagu " + head_1017.getJudul_1017() + " berhasil
dihapus");
    head_1017 = head_1017.next_1017;
    if (head_1017 != null) {
        head_1017.prev_1017 = null;
    } else {
        tail_1017 = null;
    }
}

```

Method ini digunakan untuk menghapus lagu pertama pada playlist. Jika playlist kosong maka program akan menampilkan pesan bahwa playlist kosong, sedangkan jika terdapat data maka node pertama akan dihapus dan head dipindahkan ke node berikutnya. Pointer prev pada head baru juga diubah menjadi null agar hubungan linked list tetap benar.

```

public void tampilMaju_1017() {
    if (head_1017 == null) {
        System.out.println("Playlist kosong!");
        return;
    }
    Lagu_2511531017 curr_1017 = head_1017;
    System.out.println("=== Playlist Maju ===");
    while (curr_1017 != null) {
        System.out.println(curr_1017.getJudul_1017() + " - " +
curr_1017.getPenyanyi_1017());
        curr_1017 = curr_1017.next_1017;
    }
}

```

Method ini digunakan untuk menampilkan seluruh isi playlist dari depan ke belakang menggunakan pointer next. Program melakukan traversal mulai dari head hingga node terakhir sambil menampilkan judul lagu dan nama penyanyi. Method ini menunjukkan traversal maju pada Double Linked List.

```

public void tampilMundur_1017() {
    if (tail_1017 == null) {

```

```

        System.out.println("Playlist kosong!");
        return;
    }
    Lagu_2511531017 curr_1017 = tail_1017;
    System.out.println("=== Playlist Mundur ===");
    while (curr_1017 != null) {
        System.out.println(curr_1017.getJudul_1017() + " - " +
curr_1017.getPenyanyi_1017());
        curr_1017 = curr_1017.prev_1017;
    }

```

Method ini digunakan untuk menampilkan playlist dari belakang ke depan menggunakan pointer prev. Traversal dimulai dari tail hingga menuju head sambil menampilkan seluruh data lagu. Method ini menunjukkan kemampuan traversal dua arah pada Doubly Linked List.

```

public void cariLagu_1017(String judul_1017) {
    if (head_1017 == null) {
        System.out.println("Playlist kosong!");
        return;
    }
    Lagu_2511531017 curr_1017 = head_1017;
    boolean ketemu_1017 = false;
    while (curr_1017 != null) {
        if (curr_1017.getJudul_1017().equalsIgnoreCase(judul_1017)) {
            System.out.println("Lagu ditemukan!");
            System.out.println("Judul : " + curr_1017.getJudul_1017());
            System.out.println("Penyanyi : " +
curr_1017.getPenyanyi_1017());
            ketemu_1017 = true;
            break;
        }
        curr_1017 = curr_1017.next_1017;
    }
    if (!ketemu_1017) {
        System.out.println("Lagu tidak ditemukan!");
    }
}

```

Method ini digunakan untuk mencari lagu berdasarkan judul lagu yang dimasukkan pengguna. Program akan melakukan traversal dari head dan membandingkan judul lagu menggunakan equalsIgnoreCase agar pencarian tidak sensitif terhadap huruf besar dan kecil. Jika lagu ditemukan maka data lagu akan ditampilkan, sedangkan jika tidak ditemukan program akan menampilkan pesan bahwa lagu tidak ada.

```

public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    Musik_2511531017 musik_1017 = new Musik_2511531017();
    int pilihan_1017;
    do {
        System.out.println("\n=== Playlist Musik NIM: 2511531017 ===");
        System.out.println("1. Tambah Lagu");
        System.out.println("2. Hapus Lagu Pertama");
        System.out.println("3. Lihat Playlist (Maju)");
        System.out.println("4. Lihat Playlist (Mundur)");
        System.out.println("5. Cari Lagu");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
        pilihan_1017 = input.nextInt();
        input.nextLine();
    } while (pilihan_1017 != 6);
}

```

Digunakan untuk menjalankan menu interaktif playlist musik. Program menggunakan perulangan do-while agar menu terus tampil sampai pengguna memilih keluar. Di dalam menu terdapat fitur tambah lagu, hapus lagu pertama, tampil playlist maju, tampil playlist mundur, dan pencarian lagu. Program juga menggunakan switch-case untuk menjalankan menu sesuai pilihan pengguna.

```
case 1:
    System.out.print("Judul Lagu: ");
    String judul_1017 = input.nextLine();
    System.out.print("Penyanyi: ");
    String penyanyi_1017 = input.nextLine();
    musik_1017.tambahLagu_1017(
        judul_1017,
        penyanyi_1017);
```

Case ini digunakan untuk menambahkan lagu baru ke playlist. Pengguna akan diminta memasukkan judul lagu dan nama penyanyi, kemudian data tersebut dikirim ke method tambahLagu_1017().

```
case 2:
    musik_1017.hapusLaguAwal_1017();
```

Case ini digunakan untuk menghapus lagu pertama pada playlist dengan memanggil method hapusLaguAwal_1017().

```
case 3:
    musik_1017.tampilMaju_1017();
```

Case ini digunakan untuk menampilkan playlist dari depan ke belakang menggunakan method tampilMaju_1017().

```
case 4:
    musik_1017.tampilMundur_1017();
```

Case ini digunakan untuk menampilkan playlist dari belakang ke depan menggunakan method tampilMundur_1017().

```
case 5:
    System.out.print("Masukkan judul lagu: ");
    String cari_1017 = input.nextLine();
    musik_1017.cariLagu_1017(cari_1017);
```

Case ini digunakan untuk mencari lagu berdasarkan judul yang dimasukkan pengguna. Setelah input diterima, program akan menjalankan method cariLagu_1017().

```
case 6:
    System.out.println("Program selesai");
    break;
default:
    System.out.println("Pilihan tidak valid!");
```

Case ini digunakan untuk mengakhiri program playlist musik. Saat dipilih, program akan menampilkan pesan "Program selesai". Bagian default dijalankan jika pengguna memasukkan pilihan menu yang tidak tersedia. Program akan menampilkan pesan bahwa pilihan tidak valid.

Output:

```

=== Playlist Musik NIM: 2511531017 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul Lagu: Shape of My Heart
Penyanyi: Backstreet Boys
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511531017 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul Lagu: Satu Bulan
Penyanyi: Bernadya
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511531017 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
=== Playlist Maju ===
Shape of My Heart - Backstreet Boys
Satu Bulan - Bernadya

=== Playlist Musik NIM: 2511531017 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
=== Playlist Mundur ===
Satu Bulan - Bernadya
Shape of My Heart - Backstreet Boys

=== Playlist Musik NIM: 2511531017 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu: Satu Bulan
Lagu ditemukan!
Judul : Satu Bulan
Penyanyi : Bernadya

=== Playlist Musik NIM: 2511531017 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu Shape of My Heart berhasil dihapus

```

```
=== Playlist Musik NIM: 2511531017 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu Satu Bulan berhasil dihapus

=== Playlist Musik NIM: 2511531017 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Playlist kosong!

=== Playlist Musik NIM: 2511531017 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
Playlist kosong!

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
Playlist kosong!

=== Playlist Musik NIM: 2511531017 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu: Satu Bulan
Playlist kosong!

=== Playlist Musik NIM: 2511531017 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 6
Program selesai
```

Gambar 2.6

BAB III

KESIMPULAN

3.1 Kesimpulan

Double Linked List (DLL) merupakan struktur data linear yang memiliki dua pointer pada setiap node, yaitu `next` dan `prev`, sehingga data dapat ditelusuri ke arah maju maupun mundur. Struktur data ini mempermudah proses manipulasi data seperti penambahan, penghapusan, dan traversal node dibandingkan Single Linked List. Implementasi Double Linked List menggunakan Java menunjukkan bagaimana hubungan antar node dibentuk dan dikelola melalui pointer dua arah.

Penerapan Double Linked List dapat digunakan pada berbagai program seperti playlist musik, navigasi data, dan sistem pengelolaan antrian. Dengan adanya traversal dua arah, proses pengaksesan data menjadi lebih fleksibel dan efisien. Oleh karena itu, Double Linked List menjadi salah satu struktur data yang penting untuk dipahami dalam pengembangan program dan pengolahan data.

3.2 Saran

Sebagai saran, sebaiknya penjelasan materi saat praktikum bisa lebih detail dan perlahan, supaya mahasiswa yang belum terlalu paham coding bisa mengikuti dengan baik.

DAFTAR PUSTAKA

- [1] Oracle, “LinkedList Class (Java Platform SE Documentation),” [Online].
Available: <https://docs.oracle.com/javase/8/docs/api/java/util/LinkedList.html>